

THE AI TOOLBOX · CLASS 13

KNN, SVM & a First Neural Network

Three ways to solve the **same** classification problem — and no automatic champion.

Merit Point Academy · 60 min · press **S** notes · → next ·  to comment

RECAP · WHERE WE LEFT OFF

Last class in 60 seconds

One readable tree

A chain of if-then questions you can print and challenge.

A forest votes

Many unstable trees average away some of one another's errors.

The honest score

Train and test stay separate; the test set is touched only for evaluation.

Tonight we keep the **same test discipline** and change the model's idea: neighbours, margins, then a tiny network.

AGENDA · SIX ATTENTION CHECKPOINTS

One fair race, three very different runners

- **0–8'** Warm-up + the no-champion story
- **8–18'** KNN: distance, k, and scaling
- **18–28'** SVM: margin, support vectors, kernels
- **28–38'** First neural net: layers and learning
- **38–53'** Two labs on one fixed split
- **53–60'** Interpret the table + launch your comparison

Quick question about every 6–8 minutes. Choose before the explanation; the pause is part of the lesson, not dead air.

QUICK QUESTION 1 · WARM-UP

Quick check · [tap an answer](#)

Three models score 97%, 98%, and 96% — but each used a different random train/test split. What can you honestly conclude?

The 98% model wins

The 96% model is underfitting

Nothing about which model is better — they did not face the same test

Average the three percentages and pick the closest model

THE STORY · NO PERMANENT CHAMPION

The crown kept moving

Nearest-neighbour methods won by remembering examples. Support-vector machines then dominated many small, high-dimensional problems by drawing a disciplined boundary. In 2012, a deep neural network changed image recognition by learning its own features. Yet KNN and SVM never became “wrong” — they still win on the right data.

No free lunch: averaged across every possible problem, no learning algorithm is best. On a real project, the honest response is not “use the fanciest model.” It is **test plausible models fairly.**

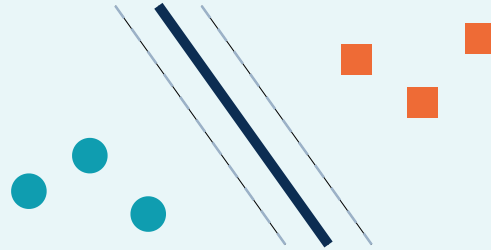
Same input, three ideas about what matters

KNN



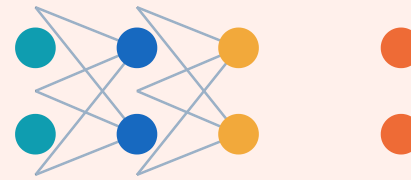
ask nearby examples

SVM



find the safest boundary

Neural net



learn useful features

teaching diagram · not measured results

One sentence each: KNN trusts nearby examples. SVM trusts the widest safe boundary. A neural network learns intermediate features that make the classes easier to separate.

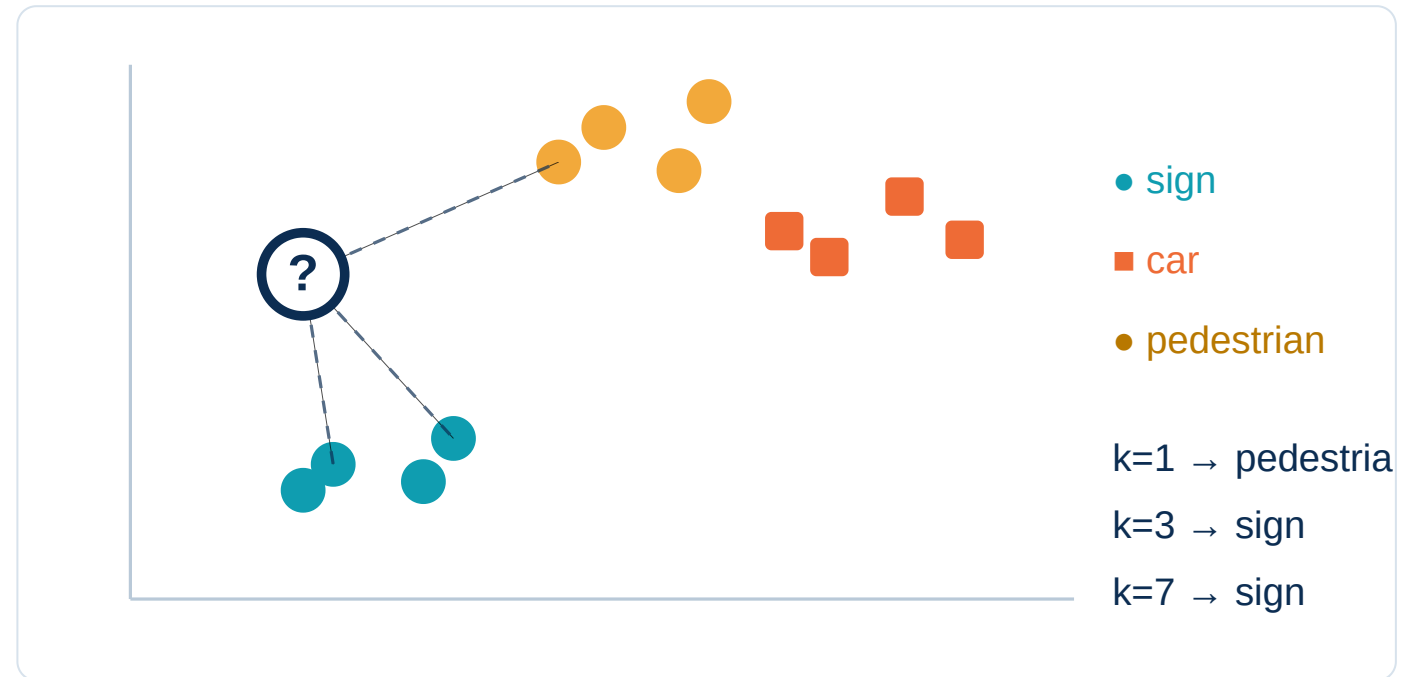
MODEL 1 · K-NEAREST NEIGHBOURS

Prediction by local vote

1. Measure the new point's distance to every labelled example
2. Keep the closest **k**
3. Let those neighbours vote

Training: almost nothing.

Prediction: the expensive part.



For the computed query shown: k=1 votes **pedestrian**; k=3 and k=7 vote **sign**. One hyperparameter changes how local the model is.

QUICK QUESTION 2 · CHOOSE K

Quick check · [tap an answer](#)

Your 1-nearest-neighbour model gets every training row correct but jumps wildly when one noisy point is added. What is the best first move?

Reduce k below 1

Try a larger odd k and validate it on held-out data

Add more decimal places to the distances

Switch the labels from words to numbers

Distance listens to the biggest unit

Raw units

feature 1 spans 1 → 10

feature 2 spans 0 → 1

nearest class: sign



After StandardScaler

both features centered at 0

both have standard deviation 1

nearest class: car

computed from the six points shown in the lesson notes

In this six-point calculation, the exact same query's nearest class flips from **sign** to **car** after standardisation. No labels changed; only the ruler did.

Pipeline rule: fit `StandardScaler` on the training rows only, then transform train and test with that fitted scaler. Putting both inside an `sklearn Pipeline` prevents leakage.

Do not just separate — leave the widest street

Boundary

A line in 2-D, a plane in 3-D, a hyperplane in more dimensions.

Margin

The empty corridor between the boundary and the nearest points.

Support vectors

The few edge cases touching that corridor. They determine the solution.

Why the margin? Many lines may classify the training points. SVM chooses the one with the most room for small measurement errors.

Quick check · [tap an answer](#)

You move one training point that sits far from the SVM margin, while every support vector stays fixed. What usually happens to the boundary?

It flips to the other class

It usually stays the same — support vectors determine the boundary

The margin becomes zero

The kernel automatically changes

When no straight line can work

Linear kernel

One flat boundary. Fast, interpretable geometry, strong when features already separate the classes.

Teaching diagram: two interlocking moons defeat it.

RBF kernel

Similarity to landmark points creates a curved boundary without explicitly building every transformed feature.

The recording computes both boundaries on the same moon data.

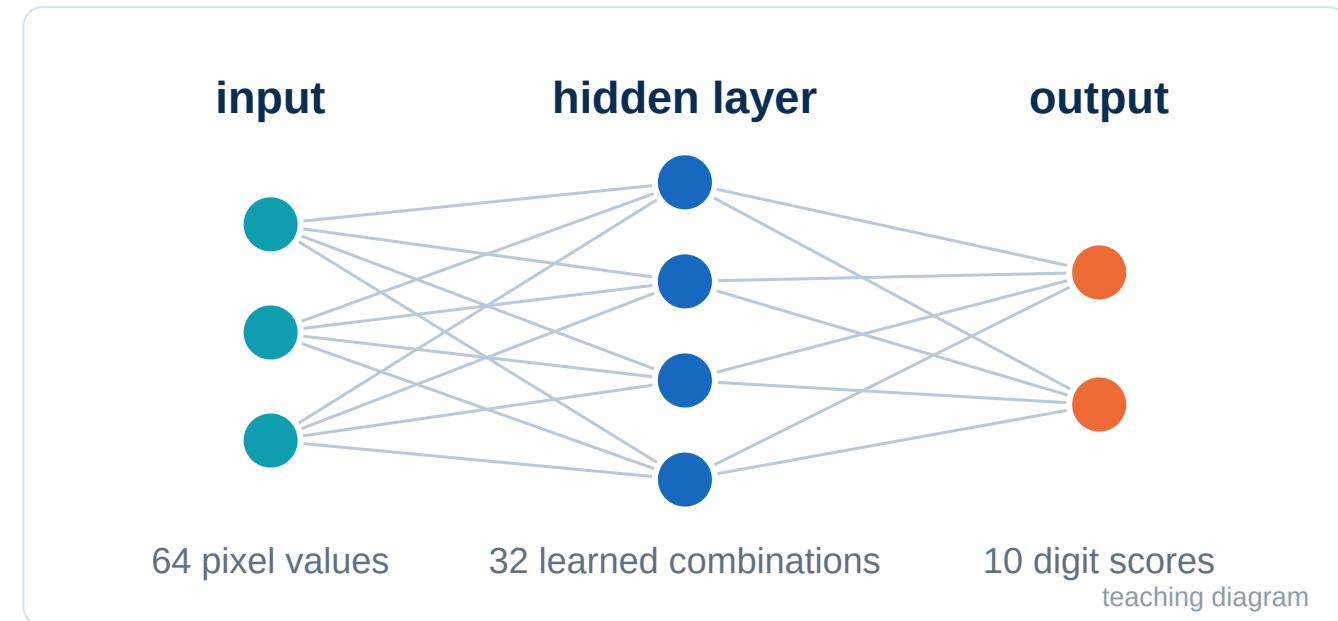
C controls how hard the model punishes training mistakes. **gamma** controls how local the bends are. Both can overfit; neither should be tuned on the final test set.

MODEL 3 · A FIRST NEURAL NETWORK

A stack of weighted feature makers

- **Input layer:** the 64 pixel values in one 8×8 digit
- **Hidden layer:** 32 learned combinations
- **Output layer:** 10 scores, one for each digit

Our “first NN” is an sklearn **MLPClassifier**: one hidden layer, not a deep vision system.



A neuron computes a weighted sum, adds a bias, then applies a non-linear activation. Without the non-linearity, stacking layers collapses back into one linear model.

QUICK QUESTION 4 · COUNT WHAT LEARNS

Quick check · tap an answer

A hidden neuron receives 64 pixel inputs and has one bias. How many trainable numbers feed that one neuron?

32

64

65

640

HOW THE NETWORK LEARNS · THE SAME LOOP AT SCALE

Forward → loss → backward → update

1 · Forward

pixels become ten scores

2 · Loss

compare scores with the true digit

3 · Backprop

assign each weight its share of error

4 · Update

nudge weights downhill

What it gains

Flexible non-linear boundaries and learned intermediate features.

What it costs

More parameters, more tuning, weaker interpretability, and often more data.

CASE STUDY · ONE DATASET, ONE SPLIT

Recognise 8×8 handwritten digits

1,797 examples

Each image is an 8×8 grid, flattened to 64 pixel-intensity features.

1,347 train

The scaler and all three models learn only from these rows.

450 test

Held out once with `random_state=13` and stratification.

Why not 42,000 MNIST rows? The lesson's idea does not require a browser to grind through a large SVC. This smaller real dataset keeps all ten classes and finishes quickly enough to experiment.

Quick check · [tap an answer](#)

Which setup makes the three-model comparison fair without leaking information from the test set?

Scale the full dataset once, then split it three different ways

Create one split; fit a separate scaler on each model's full data

Create one split; give every model a Pipeline whose scaler fits only the shared training rows

Let each model choose whichever test rows produce its best score

Write KNN from scratch — then watch k change the vote

Twelve labelled points, one unknown point, Euclidean distance, and a vote. Nothing is hidden behind an API.

```
1 from math import dist
2 from collections import Counter
3
4 train = [
5     ([1.0, 1.0], "sign"), ([1.2, 1.3], "sign"), ([1.8, 1.1], "sign"), ([2.0, 1.6], "sign"),
6     ([4.2, 4.0], "car"), ([4.5, 3.7], "car"), ([5.0, 4.4], "car"), ([5.4, 3.9], "car"),
7     ([2.7, 4.8], "pedestrian"), ([3.0, 5.2], "pedestrian"),
8     ([3.5, 4.7], "pedestrian"), ([3.7, 5.5], "pedestrian"),
9 ]
10 query = [1.0, 3.5]
11
12 ranked = sorted((dist(x, query), label, x) for x, label in train)
```



Run



Explain



Debug



Improve



Mira

Python loads on first Run

► Run the code to see the output here.

KNN vs. SVM vs. one-hidden-layer MLP

A real ten-class image dataset, one fixed split, one metric, and three sklearn Pipelines. This is the comparison table you will build for your own project.

```
1 from sklearn.datasets import load_digits
2 from sklearn.model_selection import train_test_split
3 from sklearn.pipeline import make_pipeline
4 from sklearn.preprocessing import StandardScaler
5 from sklearn.neighbors import KNeighborsClassifier
6 from sklearn.svm import SVC
7 from sklearn.neural_network import MLPClassifier
8 from sklearn.metrics import accuracy_score, confusion_matrix
9
10 X, y = load_digits(return_X_y=True)
11 X_train, X_test, y_train, y_test = train_test_split(
12     X, y, test_size=0.25, random_state=13, stratify=y)
```

[▶ Run](#)[🌟 Explain](#)[🐛 Debug](#)[⚡ Improve](#)[🤖 Mira](#)

Python loads on first Run

▶ Run the code to see the output here.

RESULTS · THE CHECKED REFERENCE RUN

SVM wins this race — by eight test images

KNN · k=3



97.33% · 12 errors

SVM · RBF



98.22% · 8 errors

MLP · 32 hidden



96.89% · 14 errors

axis starts at 94% to make small differences readable · n=450 held-out digits

Do not overclaim: “SVM scored highest on this split with these settings” is supported. “SVM is the best image model” is not. A modern convolutional network, more data, or a different task changes the contest.

QUICK QUESTION 6 · WRITE THE HEADLINE

Quick check · tap an answer

Lab 2 gives SVM 98.22% and KNN 97.33% on the same 450 test digits. Which headline is scientifically honest?

SVM is always the best classifier

SVM was 0.89 percentage points higher on this held-out split; repeat runs or cross-validation would test whether the gap is stable

KNN failed because it scored below 100%

Neural networks are obsolete because this small MLP came third

DISCUSS · CHOOSE FOR A REAL CONSTRAINT

The score is not the whole decision

Tiny dataset, instant explanation

Would you choose KNN because a prediction can point to its nearest examples?

Small, high-dimensional data

Would you choose SVM for its strong margin and compact support-vector solution?

Millions of raw images

Would you choose a neural network because learned features can scale?

Prompt: pick one deployment and defend a model using data size, prediction speed, interpretability, and expected boundary shape — not popularity.

Three models, one split — then one controlled change

1. Choose one classification dataset
2. Create **one stratified train/test split**; record the seed
3. Build KNN, SVM and a **one-hidden-layer MLP** as Pipelines
4. Reuse the exact same train and test rows
5. Add one fourth row that changes only **k, kernel, or hidden size**
6. Report accuracy plus integer error count

Deliverable

One shared Colab notebook that passes **Runtime** → **Run all**, a table with at least four rows, and a bounded two-sentence conclusion.

Success test

Every row names its configuration and uses the same preprocessing rule, train rows and test rows.

BUILD IT NOW · KEEP THE RACE FAIR

The six-line experimental contract

1. **Question:** which of three classifiers generalises best here?
2. **Data:** same X and y for every model
3. **Split:** once, stratified, seed recorded
4. **Preprocessing:** inside each Pipeline; trained on train only
5. **Metric:** chosen before reading the scores
6. **Claim:** limited to this experiment

scikit-learn

Pipeline

StandardScaler

pandas

matplotlib

If you tune k, C, gamma, or hidden-layer size, make a **validation** split or use cross-validation. The final test set is not a scoreboard you keep refreshing.

A table plus a bounded conclusion

Model	Preprocessing	Test rows	Accuracy	Errors
KNN · k=3	StandardScaler	450	0.9733	12
SVM · RBF	StandardScaler	450	0.9822	8
MLP · 32 hidden	StandardScaler	450	0.9689	14

Model conclusion: “On sklearn digits with seed 13, the RBF SVM made 8 errors, versus 12 for KNN and 14 for the small MLP. The margin is only four errors over KNN; repeated evaluation is needed before calling it stable.”

One fair comparison + one controlled variant

Protocol

- 1. One **stratified** split + recorded seed
- 2. KNN · SVM · one-layer MLP Pipelines
- 3. Same preprocessing rule + same train/test rows
- 4. One variant: change only **k**, **kernel**, or **hidden size**

Submit

Shared Colab; **Runtime** → **Run all** succeeds · two bounded conclusion sentences · one limitation + next test · AI table: **tool | used for | verified**.

Due: Sunday · Aug 23 · 12:00 PM ET
Tuesday · Aug 25 · 7:00 PM ET

Model / config	Preprocess	Train / test	Accuracy	Errors
KNN · k=...	Pipeline	... / ...	...	...
SVM · kernel=...	Pipeline	same	...	...
MLP · hidden=(...,)	Pipeline	same	...	...
Controlled variant	same	same	...	...

Rubric · 20 / 25 / 20 / 15 / 15 / 5

Data source + split + seed + reproducibility **20** · fair three-model Pipelines **25** · valid one-variable controlled change **20** · complete evidence table **15** · bounded conclusion + limitation/next test **15** · AI disclosure + submission format **5**.

Stretch: compare candidates with 5-fold CV on training only; final test never chooses the winner.

TODAY'S LINE TO REMEMBER

**“There is no best model.
There is only the best-tested claim.”**

Compare on the same unseen data. Name the conditions. Keep the conclusion smaller than the evidence.

WRAP-UP

Three models, four rules

- **KNN** compares neighbours — scale the ruler and validate k
- **SVM** maximises a margin — support vectors anchor it; kernels bend it
- **MLP** learns hidden features — powerful, parameter-hungry, less readable
- A fair bake-off uses **one split, one metric, and a bounded claim**

Exit ticket: which model would you try first on your data, and what property of the data justifies it?

Next · Class 14 — experimental design: is the winning gap real, or luck?